IKT453 · INTELLIGENT DATABASE MANAGEMENT · SPRING 2026

Epidemiological Data Warehouse

& Analytics Platform

A Multi-Backend Approach to Disease Surveillance Analytics

Using Relational, Document, and Graph Databases

Anwar Debes
Supervisor: Prof. Vladimir I. Zadorozhny
University of Agder · Faculty of Engineering and Science

The Problem

Why does the database paradigm matter for analytics?

One dataset. Three paradigms. Same questions.

Epidemiological surveillance produces **century-scale, structured time-series data** weekly case counts across diseases, states, and decades.

Traditional relational warehouses have served this role for decades, but:

- Document databases eliminate join overhead for selective queries.
- Graph databases enable traversal queries SQL cannot express naturally.
- Stream processing extends batch warehouses with real-time insight.

This project answers that question empirically.

RESEARCH QUESTION

Given the same epidemiological dataset and the same analytical queries, how do **relational, document, and graph** database implementations compare in terms of:

- data modeling complexity
- query expressiveness
- execution performance

Project Objectives

The eight deliverables, as set by the project brief

1. Design a data warehouse for a chosen application domain
2. Prepare decision support queries the warehouse must serve
3. Select data sources and perform ETL transformations
4. Implement as a relational star schema (Kimball model)
5. Provide two NoSQL alternative implementations
6. Compare relational vs NoSQL advantages and trade-offs
7. Dockerize the complete stack (DB + NoSQL + streaming)
8. Apply stream processing as an additional contribution

All 8 delivered. Objective 8 (streaming) was designated as optional extra credit in the brief.

The Dataset

Project Tycho Level 1 University of Pittsburgh

A research-quality subset of US notifiable-disease surveillance data, curated from the country's weekly disease reporting system (operating since 1888).

473,189
Weekly observations (cleaned)

96
Years (1916 – 2011)

4,947
Distinct epi-weeks

171
Locations (51 states + 120 cities)

24,878,333
Total reported cases across the corpus

Coverage

Measles, Pertussis, Polio, Diphtheria, Smallpox,
Hepatitis A, Mumps, Rubella

7 diseases at state grain · Diphtheria at city grain (1916-1947)

Raw file: 759,467 rows. After cleaning (drop zeros, nulls, non-numeric): 473,189 weekly observations.

System Architecture

Four layers, strict separation of concerns

PRESENTATION

React Dashboard · Port 3000

API LAYER

FastAPI REST · /query /compare /olap /stream /metadata · Port 8000

STORAGE

PostgreSQL 18

Star Schema

Port 15432

MongoDB 8

Bucket Pattern

Port 27017

Neo4j 2026.03

Labeled Property Graph

Port 7474 / 7687

INGESTION

Kafka KRaft (Port 9092)
ksqlDB Server (Port 8088) · ETL Pipeline (Python): Extract → Transform → Load

Source: Project Tycho Level 1 CSV (473K clean rows) → batch ETL + Kafka stream

Docker Service Topology

Eight containers, one `docker compose up`

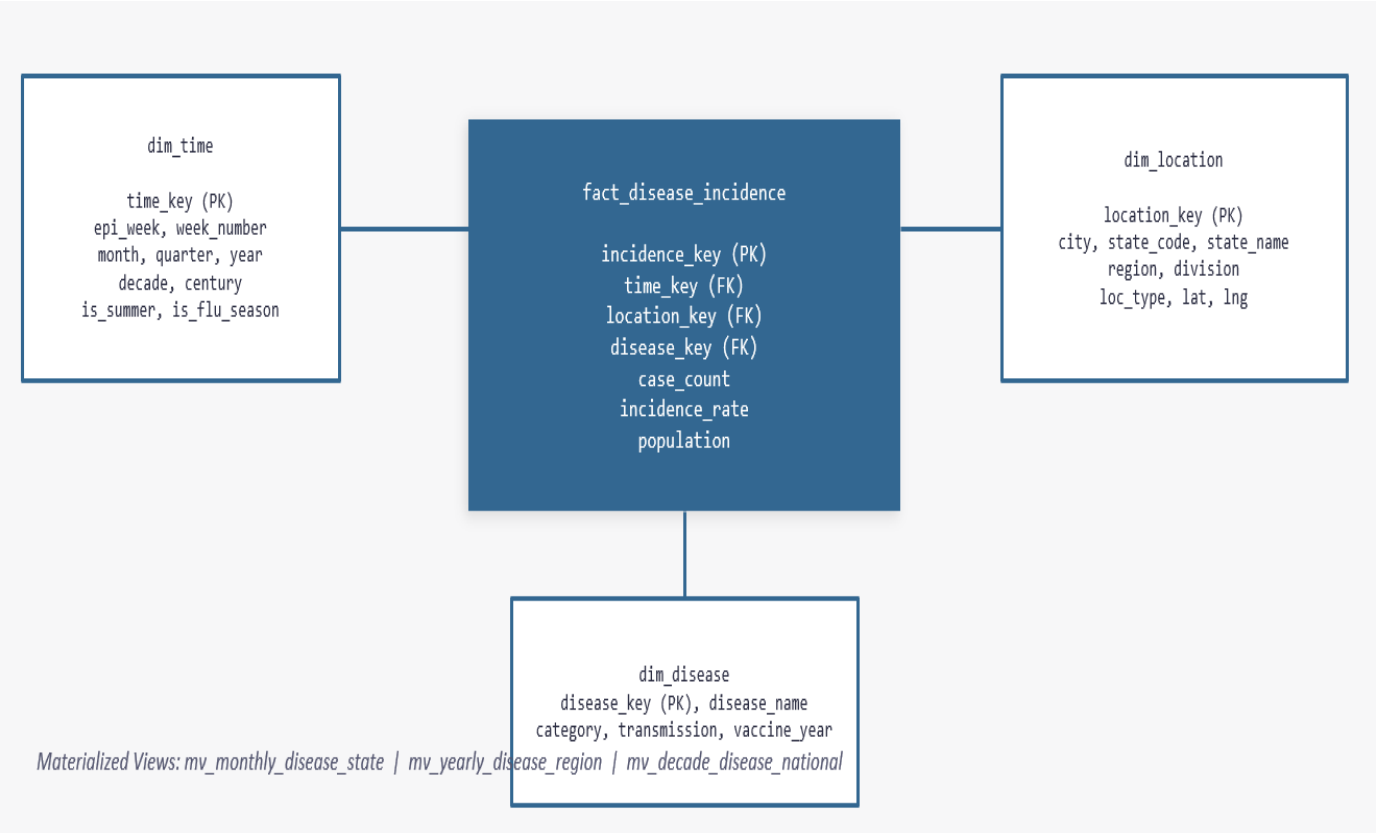
Service	Container	Port	Role
PostgreSQL 18	epid-postgres	15432	Relational star schema DW
MongoDB 8	epid-mongodb	27017	Document-based DW (Bucket Pattern)
Neo4j 2026.03	epid-neo4j	7474 / 7687	Graph-based DW with APOC
Kafka (KRaft)	epid-kafka	9092	Event-streaming broker
ksqlDB Server	epid-ksqldb-server	8088	Stream-processing engine
ksqlDB CLI	epid-ksqldb-cli		Interactive ksqlDB client
FastAPI backend	epid-backend	8000	Unified REST API
React frontend	epid-frontend	3000	Analytics dashboard

KRAFT MODE NO ZOOKEEPER

Kafka runs in KRaft mode (Raft-based internal consensus), which replaces the external ZooKeeper dependency. This reduces the total container count from 9 to 8 and is the only supported metadata mode in Kafka 4.0+. Confluent Platform 8.2 ships Kafka 4.2 and ksqlDB as a matched release.

Backend 1 PostgreSQL Star Schema

Classical Kimball dimensional model



Design highlights

- Three dimension tables + one fact table.
- Composite index on (disease_key, time_key, location_key) covers the primary star-join access pattern.
- Four materialized views pre-aggregate at progressively coarser grain: monthly-by-state, yearly-by-region, decade-national, yearly-city.
- dim_time supports drill-down: week → month → quarter → year → decade.
- Query tools used: CTEs, window functions (LAG, RANK, STDDEV), LATERAL joins, CORR aggregate.

Total size on disk: **92 MB** · Fact table: 66 MB · Mat. views: 17.4 MB · Dimensions: <1 MB · End-to-end load + index + refresh: ~163 s

Backend 2 MongoDB Bucket Pattern

Denormalized time-series documents

The problem with naïve time-series

Storing one document per weekly observation would create 473,189 small documents high overhead in index size and query latency.

Bucket Pattern solution

Group observations by (disease, location, month) into a single document containing an array of weekly measurements plus pre-computed monthly summary.

155,063	3.05	114 MB
Bucket documents	Weekly obs / bucket	Total size

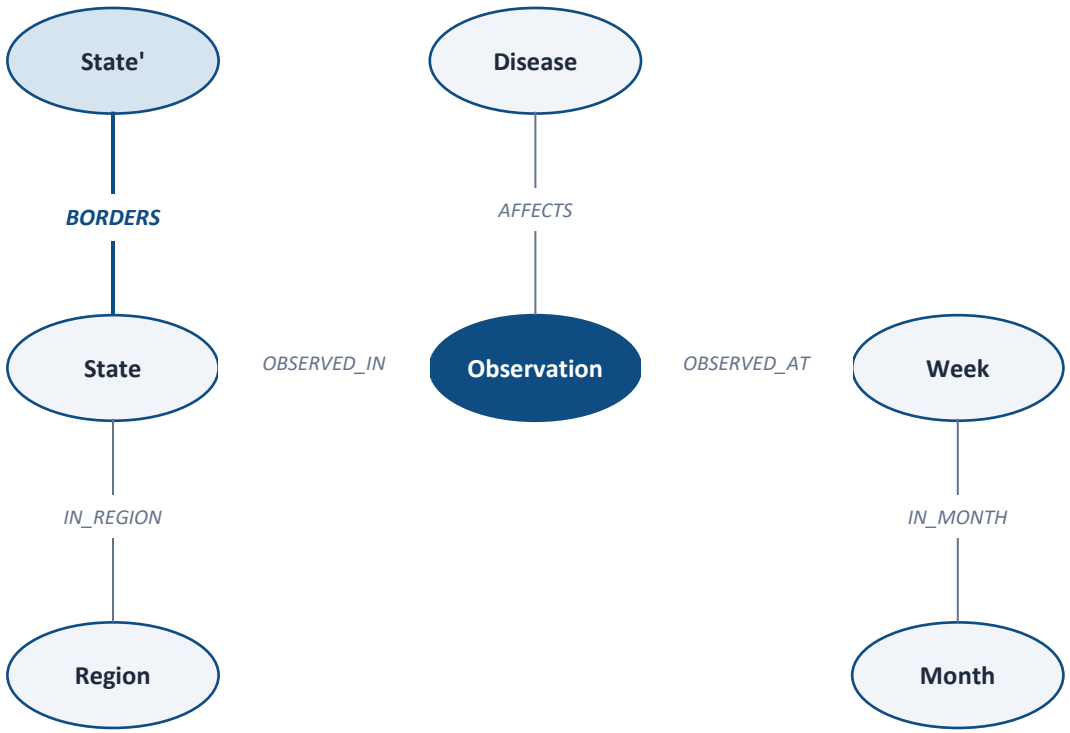
Aggregation pipeline operators used: $group · $unwind · $match · $setWindowFields · $stdDevPop · $merge

BUCKET DOCUMENT

```
{
  "disease": { "name": "MEASLES", "category": "Viral",
               "vaccine_year": 1963 },
  "location": { "state_code": "CA",
               "state_name": "CALIFORNIA",
               "region": "West" },
  "time_bucket": { "year": 1960, "month": 3,
                   "decade": 1960 },
  "weekly_observations": [
     { "epi_week": 196009, "cases": 1247, ... },
     { "epi_week": 196010, "cases": 1389, ... }
  ],
  "monthly_summary": {
     "total_cases": 5604,
     "avg_incidence_rate": 8.91,
     "peak_weekly_cases": 1512,
     "observation_count": 4
  }
}
```

Backend 3 Neo4j Graph Model

Labeled property graph with BORDERS adjacency



Graph composition

473,189 Observation nodes

139,683 MonthlyAggregate nodes

6,774 dimension + hierarchy nodes

214 BORDERS edges (48 contiguous states)

1,845,584 relationships total

Why the graph earns its keep

- BORDERS relationships enable spatial spread analysis with no natural SQL equivalent.
- Index-free adjacency: constant-time traversal per hop.
- City-level diphtheria linked to parent state via LOCATED_IN_STATE.

ETL Pipeline

Single-pass extraction feeding three backend loaders

SOURCE

Tycho L1 CSV

759K raw rows
473K after cleaning

→

EXTRACT

extract.py

pandas DataFrames
All reference CSVs

→

TRANSFORM

transform.py

Clean + derive
Surrogate keys
Three output shapes

→

LOAD (SQL)

load_postgres.py

COPY protocol
psycopg2

LOAD (DOC)

load_mongo.py

insert_many
batch = 10,000

LOAD (GRAPH)

load_neo4j.py

UNWIND batches
size = 5,000

Reproducibility switch: a single environment variable, `TYCHO_DATA_FILE`, toggles the pipeline between the real Project Tycho Level 1 file and the synthetic benchmark file with no code changes. The synthetic generator is calibrated on the real data's statistical characteristics (base rates, seasonality, vaccine effects).

Streaming Layer Kafka + ksqlDB

Four-stage continuous pipeline ending at a live dashboard

PYTHON

Producer

Reads CSV,
publishes JSON
to raw-disease-events

→

KSQLDB

1. Ingest stream

disease_events_stream
(JSON schema)

→

KSQLDB FILTER

2. Validate

disease NOT NULL
cases ≥ 0

→

KSQLDB TABLE

3. Windowed aggregate

weekly_national_summary
SUM by (disease, epi_week)

→

KSQLDB FILTER

4. Anomaly stream

incidence_rate > 50
→ anomaly_alerts

SERVER-SENT EVENTS (SSE)

FastAPI consumes anomaly_alerts and weekly_national_summary → streams live events to the **React dashboard** via the `/api/v1/stream/events` SSE endpoint. Events appear in the UI with no polling.

Persistent queries, not polling. Each stage is a ksqlDB `CREATE STREAM` / `CREATE TABLE` statement that runs continuously. Incoming events cascade through the pipeline in real time.

Unified REST API

One façade over three backends and the comparison engine

Method	Endpoint	Purpose
GET	`/api/v1/health`	Backend connectivity status
GET	`/api/v1/queries`	List all 13 queries with metadata
POST	`/api/v1/query`	Execute query on one backend
POST	`/api/v1/compare`	Run on all three backends concurrently
POST	`/api/v1/olap/{op}`	OLAP operations (slice, dice, …)
GET	`/api/v1/metadata/{backend}`	Schema introspection
POST	`/api/v1/batch/refresh-summaries`	Refresh materialized views
GET	`/api/v1/stream/events`	SSE real-time Kafka feed

THE COMPARISON ENGINE

`/api/v1/compare`

One logical query ID → dispatched to all three backends in parallel using a ThreadPoolExecutor (3 workers).

Each worker returns:

- `results` normalized row set

`execution_time_ms` measured locally

`query_text` the native SQL / pipeline / Cypher

This is why the project can claim an empirical comparison rather than a theoretical one the dashboard lets you run the same question against the three paradigms side by side and see the execution time and the native query text.

Decision Support Queries

Ten cross-backend, three graph-only 13 in total

Cross-backend (PG / Mongo / Neo4j)

ID	Analytical purpose	Technique
Q1	Cases by disease and decade	*roll-up*
Q2	Per-state cases for one disease in a year	*slice*
Q3	Top-10 states by disease over a year range	*ranked dice*
Q4	Seasonal pattern by calendar month	*roll-up*
Q5	Year-over-year change in cases per state	*window*
Q6	Disease co-occurrence across states	*correlation*
Q7	Geographic spread ranking	*aggregation*
Q8	Vaccination impact pre vs. post	*LATERAL / CALL*
Q9	Anomaly detection by z-score	*stats*
Q10	Normalized cross-disease trend	*normalization*

Graph-only (Neo4j BORDERS adjacency)

ID	Analytical purpose	Technique
Q11	Cross-border disease spread lag	*graph traversal*
Q12	Per-state disease composition profile	*graph aggregate*
Q13	Disease centrality / coverage fraction	*graph stats*

Why three graph-only queries?

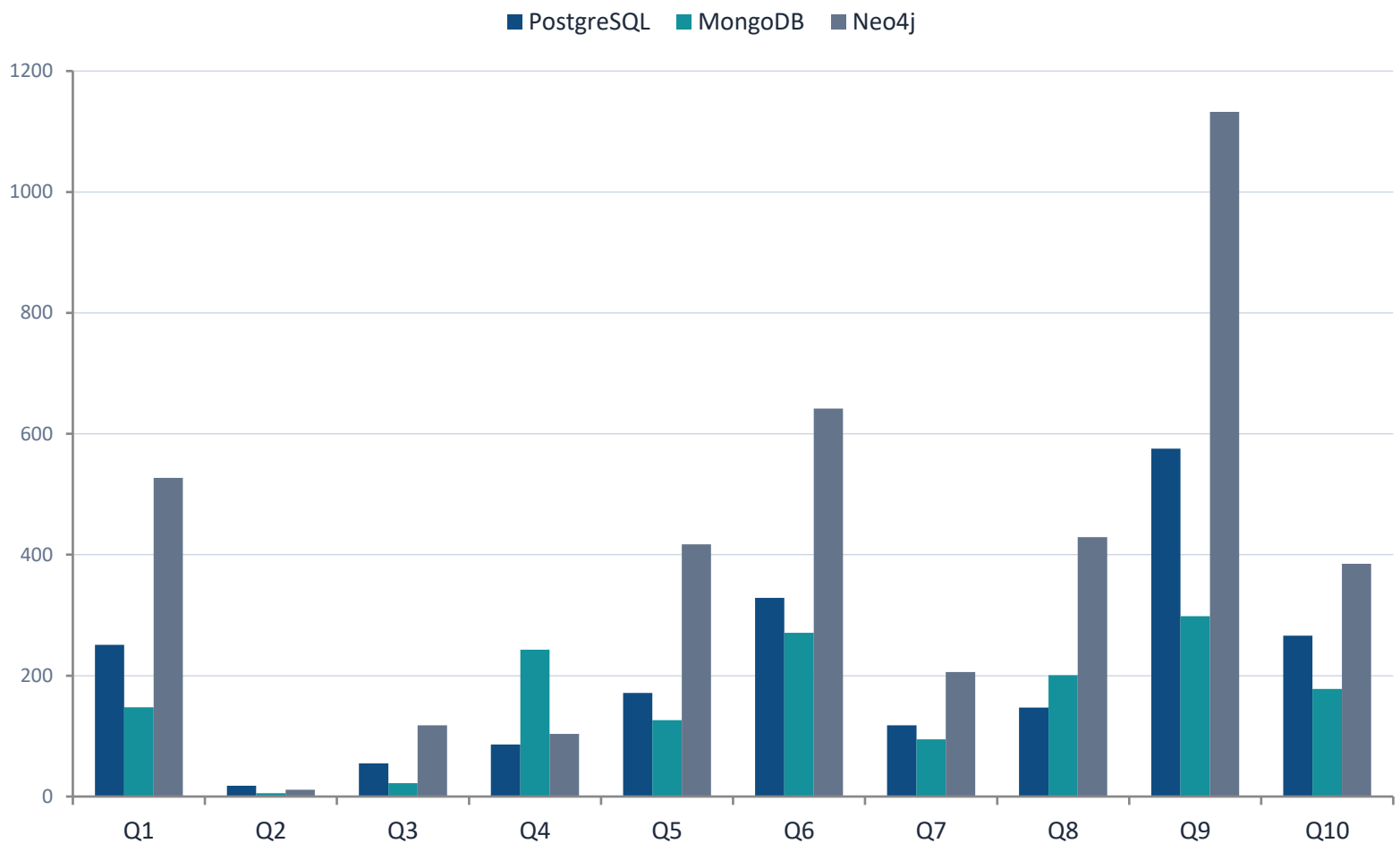
These queries rely on the BORDERS relationship between states a first-class edge in the graph model with no natural equivalent in SQL joins or MongoDB lookups.

Emulating them in SQL would require recursive CTEs and self-joins, which are syntactically awkward and computationally expensive for multi-hop traversals.

Key design principle: *each implementation exploits its backend's strengths rather than transliterating. PostgreSQL uses LATERAL joins, MongoDB uses `$setWindowFields`, Neo4j uses CALL subqueries.*

Query Performance Comparison

Median of 5 timed runs each backend in its native language



Reading the results

MongoDB wins 8 of 10 (Q1, Q2, Q3, Q5, Q6, Q7, Q9, Q10) selective queries benefit from join-free bucket documents.

PostgreSQL wins Q4, Q8.

Q4 roll-up needs fine-grain week data (MongoDB must $unwind). Q8's LATERAL join runs pre/post averages in a single scan.

Neo4j slowest on all 10.

Expected Neo4j's value is in traversal patterns, not pure OLAP aggregates. Its real contribution is Q11–Q13 (next slide).

Graph-Exclusive Analysis (Q11 – Q13)

Queries that cannot be expressed naturally in SQL or Mongo

Q11 Cross-border disease spread lag

For every disease and every ordered pair of neighbouring states that both reported the disease, find the earliest month the disease appeared in each and compute the lag. Traverses BORDERS edges and crosses hundreds of millions of candidate observation pairs.

MEDIAN EXECUTION

19,811 ms

Highest cost

Q12 Per-state disease composition profile

For each state, compute the percentage breakdown of reported cases across the 8 diseases. Produces a profile vector per state that could drive similarity or clustering analyses.

MEDIAN EXECUTION

252 ms

Low cost

Q13 Disease centrality / geographic coverage

For each disease, count the distinct (state, year) combinations with significant case counts, normalised against the full 51 × 96 = 4,896 space. A coverage fraction across the corpus.

MEDIAN EXECUTION

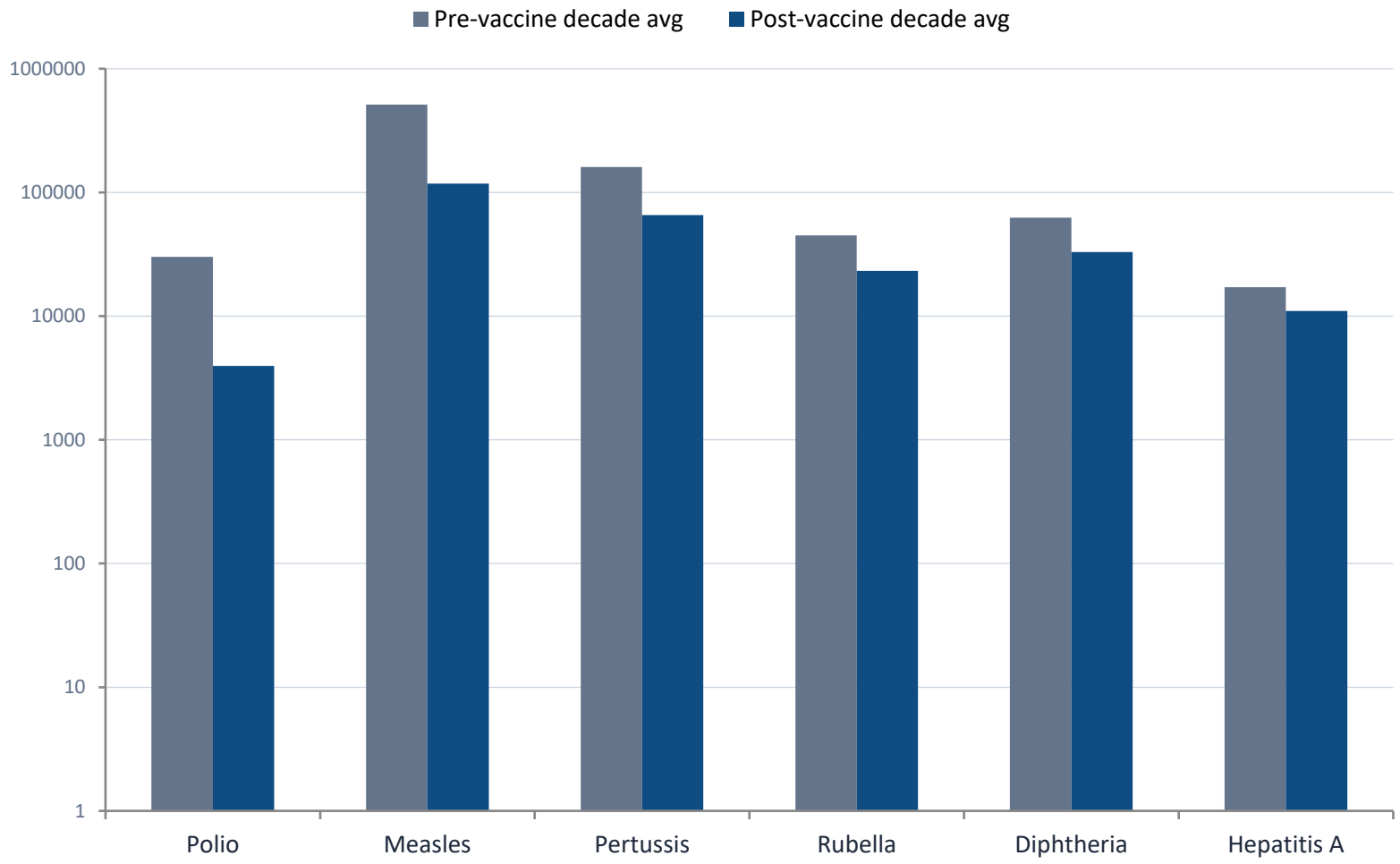
165 ms

Low cost

Q11's 19.8 s median reflects the shape of the computation (a near-Cartesian product of observation pairs across bordering states), not the engine. This is the expected cost profile for a non-trivial graph-pattern query and precisely why it cannot be answered efficiently in SQL or MongoDB.

Analytical Insight Vaccination Impact (Q8)

Average annual cases: decade before vs. decade after vaccine introduction



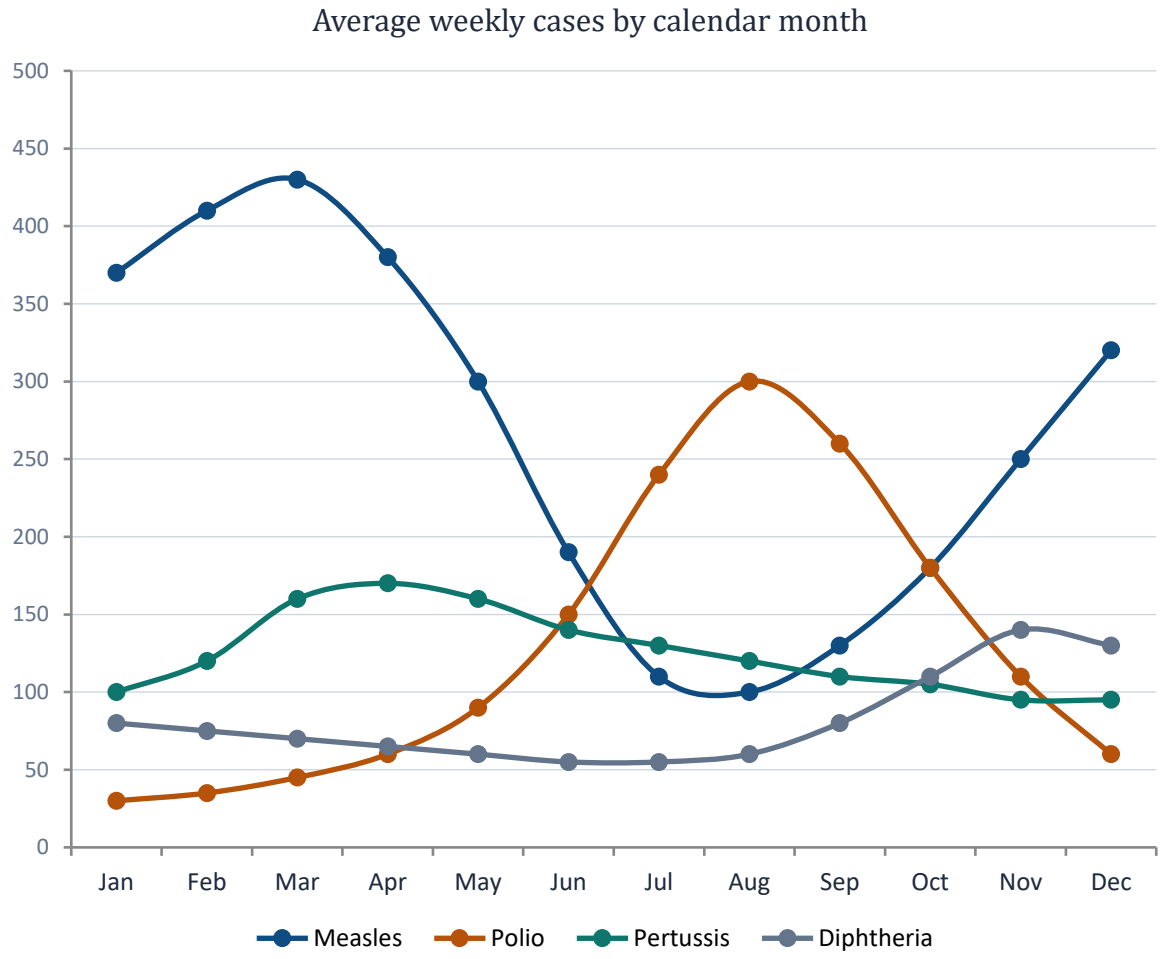
Reduction after vaccine

Polio	**−86.83%**
Measles	**−77.02%**
Pertussis	**−58.97%**
Rubella	**−48.35%**
Diphtheria	**−47.21%**
Hepatitis A	**−35.76%**

Mumps and smallpox excluded their vaccine years fall outside the pre/post windows available in the Tycho Level 1 dataset.

Seasonal Patterns & Decade Trends (Q4, roll-up)

Distinct disease signatures surfaced by roll-up across time



Decade pivot selected diseases

Decade	Measles	Polio	Smallpox
1920s	822K	7.5K	74.9K
1930s	4.86M	76.6K	148.2K
1940s	4.71M	164K	9.7K
1950s	5.20M	246K	77
1960s	2.65M	6.3K	0
1970s	347K	0	0
1980s	50.5K	0	0
1990s	33.9K	0	0
2000s	143	0	0

Vaccine-driven collapses are visible in the table: measles after 1963, polio after 1955, smallpox after 1960.

Discussion Backend Trade-offs

Each paradigm wins for a different analytical shape

PostgreSQL

Structured OLAP

STRENGTHS

- Strongest all-rounder.
- CTEs, window functions, LATERAL joins, CORR all native.
- Materialized views cut repeated work.
- Star schema gives referential integrity + compact storage.

TRADE-OFFS

- No natural variable-length traversal.
- Cross-table joins pay a cost on narrow queries.

MongoDB

Selective filtering

STRENGTHS

- Fastest on Q2, Q3 direct index seek, no joins.
- Bucket Pattern well-matched to time-series.
- $setWindowFields brings window ops into the pipeline.
- Schema flexibility for reference data.

TRADE-OFFS

- Q6: no native correlation operator.
- Q4 roll-up pays $unwind cost.
- Denormalization: ~24% storage premium.

Neo4j

Traversal patterns

STRENGTHS

- Q11–Q13 impossible elsewhere.
- Index-free adjacency: O(1) per hop.
- BORDERS edges = first-class spatial model.
- Natural fit for hierarchy (city → state).

TRADE-OFFS

- Slowest on all 10 standard OLAP queries.
- Time-hierarchy chain adds per-aggregate overhead.

Takeaway: there is no single "winner." Backend choice should follow the analytical shape, not the other way around.

Conclusion & Future Work

What the project demonstrates, and where it could go next

Delivered

- A complete multi-backend data warehouse on real Project Tycho Level 1 data (473,189 rows, 1916–2011).
- All three paradigms implemented natively not transliterated with backend-specific query idioms.
- 13 decision support queries, 5 OLAP operations, cross-backend comparison engine.
- Kafka + ksqlDB streaming layer with 4 persistent queries and live SSE feed to the dashboard.
- Full dockerization: 8 containers, one `docker compose up`.
- React dashboard deployed live on Vercel.

Future work

- Add a time-series backend (TimescaleDB / InfluxDB) as a fourth paradigm for comparison.
- Extend the graph with demographic and mobility edges, to enable additional graph-specific analyses beyond Q11–Q13.
- Scale the measurement protocol to larger synthetic loads and characterize how the relative performance curve evolves past 473K rows.
- Real Project Tycho Level 2 data integration.
- Historical query performance logging.

HEADLINE FINDING

A single real epidemiological dataset can be modelled in three fundamentally different paradigms **and each carries distinct analytical advantages.** PostgreSQL for structured OLAP; MongoDB for selective filtered queries; Neo4j for traversal patterns over state borders. The comparison engine makes the trade-offs measurable rather than theoretical.

DEMO

Live deployed dashboard

WEB APP

https://idm-one.vercel.app/

Source code

github.com/AnwarDebes/IDM

Thank you questions welcome.